

Lời cam đoan

Nhóm nghiên cứu cam đoan nội dung và kết quả trong báo cáo được xây dựng từ quá trình khảo sát, thiết kế, cài đặt và kiểm thử của nhóm. Các thư viện, tài liệu kỹ thuật và mã nguồn tham khảo được ghi nguồn phù hợp. Những chức năng chưa có luồng chạy hoặc chưa được kiểm thử được trình bày ở trạng thái đang hoàn thiện hoặc hướng phát triển, không được xem là kết quả đã hoàn thành.

Nhóm nghiên cứu

Lời cảm ơn

Nhóm nghiên cứu xin gửi lời cảm ơn đến giảng viên hướng dẫn, khoa chuyên môn và các cá nhân đã hỗ trợ trong quá trình thực hiện đề tài.

Nội dung phần này sẽ được hoàn thiện ở giai đoạn cuối của báo cáo.

Tóm tắt

Đề tài nghiên cứu và xây dựng NetworkTools, một ứng dụng desktop hỗ trợ quản lý tập trung và tự động hóa một số tác vụ cấu hình thiết bị mạng trong môi trường học tập, phòng lab. Hệ thống sử dụng Qt Quick/QML cho giao diện, PyQt6 làm cầu nối nghiệp vụ, SQLite để lưu thiết bị và cấu hình mong muốn, cùng các worker Python để thu thập dữ liệu, sinh lệnh từ Jinja2 template và triển khai cấu hình đến thiết bị.

Phiên bản được khảo sát đã hỗ trợ quản lý thiết bị, nhập danh sách từ tệp, ping, duy trì phiên kết nối theo tab, sao lưu running-config và đồng bộ một phần interface/OSPF. Các luồng DHCP, Static Routing, OSPF, EIGRP và NAT/PAT đã có giao diện và backend; DHCP, Routing và NAT có cơ chế xem trước và đẩy cấu hình. Interface và ACL đã có giao diện cùng lớp lưu trữ nhưng chưa hoàn thiện luồng triển khai cấu hình trong runtime chính. VLAN, BGP, VRF và topology automation mới ở mức schema, template hoặc mã thử nghiệm.

Bộ kiểm thử tự động bao phủ persistence, hợp đồng UI, tải QML và an toàn dev-mode. Tại thời điểm rà soát, 28 trong 30 test chức năng được phát hiện đã vượt qua; hai test OSPF/EIGRP cho thấy sự không đồng nhất giữa tên bảng mà backend sử dụng và schema hiện hành. Kết quả này xác nhận nền tảng tích hợp của hệ thống, đồng thời chỉ ra các nội dung phải hoàn thiện trước khi đánh giá cuối cùng.

Từ khóa: Python, PyQt6, QML, SQLite, tự động hóa mạng, Cisco IOS, DHCP, định tuyến, ACL, NAT.

Abstract

This report presents the design and implementation of NetworkTools, a PyQt6/QML desktop application for centralized network-device management and configuration automation in laboratory environments. The current system integrates a QML user interface, a Python bridge, SQLite persistence, Jinja2-based configuration rendering, and network workers for selected Cisco IOS workflows. Implemented areas include device management, configuration backup and partial synchronization, DHCP, static routing, OSPF, EIGRP, ACL persistence, and NAT/PAT. Preview-and-push workflows are currently available for DHCP, routing, and NAT, while interface and ACL deployment remain incomplete. Automated tests cover data persistence, UI contracts, QML loading, and development-mode safety. At the time of review, 28 of 30 discovered functional tests passed; two routing contract tests exposed a schema mismatch that must be resolved before the final evaluation. The project provides an extensible foundation for network-automation research while retaining clear limitations in credential protection, rollback, multi-vendor support, and large-scale validation.

Keywords: network automation, PyQt6, QML, SQLite, Cisco IOS, DHCP, routing, ACL, NAT.

Danh mục từ viết tắt

Ký hiệu	Tên tiếng Anh	Diễn giải
ACL	Access Control List	Danh sách kiểm soát truy cập
API	Application Programming Interface	Giao diện lập trình ứng dụng
CLI	Command-Line Interface	Giao diện dòng lệnh
CRUD	Create, Read, Update, Delete	Các thao tác dữ liệu cơ bản
DHCP	Dynamic Host Configuration Protocol	Giao thức cấp phát cấu hình IP động
EIGRP	Enhanced Interior Gateway Routing Protocol	Giao thức định tuyến nội vùng nâng cao
GUI	Graphical User Interface	Giao diện đồ họa
NAT	Network Address Translation	Biên dịch địa chỉ mạng
OSPF	Open Shortest Path First	Giao thức định tuyến trạng thái liên kết
PAT	Port Address Translation	Biên dịch địa chỉ theo cổng
QML	Qt Modeling Language	Ngôn ngữ khai báo giao diện của Qt
RESTCONF	REST Configuration Protocol	Giao thức quản trị cấu hình dựa trên HTTP
SSH	Secure Shell	Giao thức truy cập từ xa có mã hóa
VLAN	Virtual Local Area Network	Mạng LAN ảo
VRF	Virtual Routing and Forwarding	Phân tách bảng định tuyến logic

Mục lục

Lời cam đoan	i
Lời cảm ơn	iii
Tóm tắt	v
Abstract	vii
Danh mục từ viết tắt	ix
Danh mục hình	xv
Danh mục bảng	xvii
1 Tổng quan đề tài	1
1.1 Bối cảnh và lý do chọn đề tài	1
1.2 Bài toán nghiên cứu	1
1.3 Mục tiêu nghiên cứu	1
1.3.1 Mục tiêu tổng quát	1
1.3.2 Mục tiêu cụ thể	1
1.4 Đối tượng và phạm vi nghiên cứu	2
1.5 Phương pháp nghiên cứu	2
1.6 Đóng góp của đề tài	2
1.7 Cấu trúc báo cáo	2
2 Cơ sở lý thuyết và công nghệ	3
2.1 Quản lý cấu hình và tự động hóa mạng	3
2.1.1 Preview, apply và verify	3
2.2 Giao thức truy cập và quản trị	3
2.3 Các nghiệp vụ mạng trong phạm vi	3
2.3.1 Interface và DHCP	3
2.3.2 Định tuyến	4
2.3.3 ACL và NAT/PAT	4
2.4 Nền tảng phần mềm	4
2.5 Chiến lược kiểm thử	4

3	Phân tích và thiết kế hệ thống	5
3.1	Tác nhân và ca sử dụng	5
3.2	Yêu cầu chức năng	5
3.3	Yêu cầu phi chức năng	6
3.4	Kiến trúc tổng thể	6
3.5	Luồng dữ liệu cốt lõi	7
3.5.1	Quản lý và đồng bộ thiết bị	7
3.5.2	Lưu cấu hình mong muốn	7
3.5.3	View và Push	7
3.6	Thiết kế cơ sở dữ liệu	8
3.7	Thiết kế giao diện	8
3.8	Thiết kế an toàn và xử lý lỗi	8
4	Xây dựng phần mềm	9
4.1	Môi trường phát triển	9
4.2	Tổ chức mã nguồn	9
4.3	Khởi tạo ứng dụng và QML bridge	9
4.4	Quản lý thiết bị, kết nối và đồng bộ	10
4.5	Interface Router	10
4.6	DHCP	10
4.7	Routing	10
4.7.1	Static route	10
4.7.2	OSPF	10
4.7.3	EIGRP	11
4.8	ACL	11
4.9	NAT/PAT	11
4.10	Tiện ích hệ thống	11
4.11	Thành phần chưa tích hợp	11
5	Thử nghiệm và đánh giá	13
5.1	Mục tiêu và nguyên tắc	13
5.2	Môi trường rà soát mã nguồn	13
5.3	Kết quả kiểm thử tự động hiện tại	13
5.4	Kịch bản thử nghiệm lab đề xuất	14
5.5	Phương pháp xác minh	15
5.6	Đánh giá hiệu năng	15

5.7	Đánh giá tổng hợp	15
6	Kết luận và hướng phát triển	17
6.1	Kết quả đạt được	17
6.2	Ý nghĩa của đề tài	18
6.3	Hạn chế	18
6.4	Hướng phát triển	18
	Tài liệu tham khảo	19
A	Hướng dẫn cài đặt và chạy	21
A.1	Yêu cầu	21
A.2	Khởi tạo và chạy ứng dụng	21
A.3	Chạy kiểm thử	21
A.4	Biên dịch báo cáo	21
B	Cấu trúc dự án và ánh xạ module	23
B.1	Ánh xạ chức năng chính	24
C	Danh sách kiểm thử và minh chứng cần hoàn thiện	25
C.1	Nhóm test tự động	25
C.2	Minh chứng cần chèn vào bản cuối	25

Danh sách hình vẽ

3.1	Kiến trúc runtime của NetworkTools	6
-----	--	---

Danh sách bảng

3.1	Yêu cầu chức năng và trạng thái hiện tại	5
3.2	Phân nhóm schema SQLite	8
5.1	Tổng hợp kết quả test tại thời điểm rà soát	14
5.2	Kịch bản lab tối thiểu	14
5.3	Biểu mẫu đo hiệu năng cần hoàn thiện	15
6.1	Đối chiếu mục tiêu và trạng thái	17

1 Tổng quan đề tài

1.1 Bối cảnh và lý do chọn đề tài

Trong phòng thực hành mạng, người quản trị thường lặp lại các bước đăng nhập thiết bị, kiểm tra trạng thái, nhập lệnh cấu hình và lưu kết quả. Khi số thiết bị hoặc số nhóm cấu hình tăng, cách làm thủ công gây khó khăn cho việc theo dõi cấu hình mong muốn, dễ phát sinh sai cú pháp và thiếu một nơi lưu trữ thống nhất. Tự động hóa không thay thế kiến thức mạng, nhưng có thể chuẩn hóa các thao tác lặp, tạo bước xem trước và lưu dấu trạng thái thực thi.

1.2 Bài toán nghiên cứu

Đề tài đặt ra bài toán xây dựng ứng dụng desktop có khả năng quản lý danh sách thiết bị và hỗ trợ quy trình khép kín:

Quản lý thiết bị → Kết nối/thu thập → Lưu trạng thái → Tạo cấu hình mong muốn → Preview → Push → Cập nhật kết quả.

1.3 Mục tiêu nghiên cứu

1.3.1 Mục tiêu tổng quát

Xây dựng nền tảng phần mềm desktop phục vụ quản lý tập trung và tự động hóa một số tác vụ cấu hình router Cisco IOS trong môi trường học tập và thực nghiệm.

1.3.2 Mục tiêu cụ thể

- Xây dựng giao diện QML và cầu nối nghiệp vụ bằng PyQt6.
- Quản lý thiết bị, backup và dữ liệu cấu hình mong muốn bằng SQLite.
- Hỗ trợ kết nối, thu thập và đồng bộ một phần trạng thái thiết bị.

- Xây dựng luồng View & Push cho DHCP, Routing và NAT.
- Xây dựng persistence cho Interface và ACL làm nền tảng hoàn thiện push.
- Kiểm thử logic dữ liệu, hợp đồng UI, QML và cơ chế dev-mode.

1.4 Đối tượng và phạm vi nghiên cứu

Đối tượng chính là router Cisco IOS trong lab, tập trung vào interface, DHCP, định tuyến tĩnh, OSPF, EIGRP, ACL và NAT/PAT. Phần mềm sử dụng Python 3.11 trở lên, PyQt6/QML, SQLite, Jinja2 và các thư viện kết nối mạng. Các nội dung quản trị quy mô doanh nghiệp, độ sẵn sàng cao, phân quyền hoàn chỉnh, mã hóa secret, switching end-to-end, firewall và topology automation chưa thuộc kết quả hiện tại.

Lưu ý phạm vi. Các template MikroTik, schema VLAN/VRF và mã thử nghiệm trong vùng di sản chưa đủ để kết luận hỗ trợ end-to-end. Báo cáo chỉ xem chúng là nền tảng hoặc hướng phát triển.

1.5 Phương pháp nghiên cứu

Nhóm sử dụng các bước: khảo sát nghiệp vụ và lệnh IOS; phân tích yêu cầu; thiết kế kiến trúc và schema; cài đặt theo module; kiểm thử tự động; thử nghiệm trên dev-mode và dự kiến đối chứng trên EVE-NG/GNS3; đánh giá giới hạn dựa trên minh chứng thu được.

1.6 Đóng góp của đề tài

Đóng góp chính là một nền tảng tích hợp UI–database–worker, cơ chế biểu diễn cấu hình pending/applied, bước preview trước khi triển khai và dev-mode ngăn mở session thật trong kiểm thử. Kiến trúc module cho phép tiếp tục bổ sung controller mới mà không thay đổi toàn bộ giao diện.

1.7 Cấu trúc báo cáo

Báo cáo gồm sáu chương: tổng quan; cơ sở lý thuyết; phân tích và thiết kế; xây dựng phần mềm; thử nghiệm và đánh giá; kết luận và hướng phát triển.

2 Cơ sở lý thuyết và công nghệ

2.1 Quản lý cấu hình và tự động hóa mạng

Một hệ thống tự động hóa cấu hình cần phân biệt trạng thái hiện tại của thiết bị, trạng thái mong muốn do người dùng khai báo và tập thay đổi đang chờ áp dụng. Trong NetworkTools, dữ liệu SQLite giữ vai trò nguồn trạng thái mong muốn; worker thu thập bản ghi pending, sinh lệnh, triển khai và cập nhật trạng thái sau khi nhận kết quả.

2.1.1 Preview, apply và verify

Preview cho phép người dùng đọc lệnh dự kiến trước khi gửi. Apply là bước thực thi trên thiết bị hoặc mô phỏng có kiểm soát trong dev-mode. Verify phải dùng dữ liệu trả về hoặc lệnh show để xác nhận thay đổi. Phiên bản hiện tại mới cập nhật trạng thái dựa trên báo cáo worker; cơ chế xác minh sau push và rollback vẫn cần hoàn thiện.

2.2 Giao thức truy cập và quản trị

SSH cung cấp kênh quản trị có mã hóa và là phương thức ưu tiên. Telnet được giữ cho mục đích tương thích lab nhưng không phù hợp với môi trường thật. Một số worker có nhánh RESTCONF; dự án cũng khai báo thư viện NETCONF, tuy nhiên mức hỗ trợ chưa đồng đều giữa các module.

2.3 Các nghiệp vụ mạng trong phạm vi

2.3.1 Interface và DHCP

Interface là điểm liên kết giữa địa chỉ L3, dịch vụ DHCP helper, ACL và NAT role. DHCP gồm pool cấp phát, dải địa chỉ loại trừ và relay/helper. Dữ liệu cần được kiểm tra định dạng IPv4, subnet mask và quan hệ khóa ngoại.

2.3.2 Định tuyến

Static route mô tả mạng đích, mask, next-hop và administrative distance. OSPF tổ chức theo process, area, network và interface. EIGRP tổ chức theo autonomous system, network, interface và các chính sách tùy chọn. Các cấu hình có cấu trúc parent–child được ánh xạ sang nhiều bảng SQLite.

2.3.3 ACL và NAT/PAT

ACL là danh sách rule có thứ tự, gồm hành động và điều kiện nguồn/đích; binding gắn ACL vào interface theo chiều in/out. NAT/PAT ánh xạ địa chỉ hoặc cổng và có thể phụ thuộc ACL, pool, interface role hay route-map.

2.4 Nền tảng phần mềm

Python cung cấp lớp nghiệp vụ và worker. Qt Quick/QML mô tả giao diện; PyQt6 đưa các đối tượng Python vào QML qua context property và signal/slot. SQLite phù hợp với ứng dụng desktop cục bộ, còn Jinja2 tách cú pháp lệnh khỏi logic điều phối. Netmiko, Nornir, ncclient và Requests phục vụ các phương thức tương tác thiết bị trong từng worker. Tài liệu Python và Netmiko được dùng làm nguồn tham khảo kỹ thuật [1, 2].

2.5 Chiến lược kiểm thử

Đề tài kết hợp unit/contract test cho persistence, smoke test cho QML, kiểm tra hợp đồng giữa QML và slot, cùng test an toàn dev-mode. Thử nghiệm lab là lớp kiểm chứng cuối nhằm đánh giá cú pháp lệnh và hành vi của thiết bị thật/mô phỏng.

3 Phân tích và thiết kế hệ thống

3.1 Tác nhân và ca sử dụng

Tác nhân chính là người quản trị hoặc sinh viên phụ trách phòng lab. Các ca sử dụng gồm quản lý thiết bị, kiểm tra kết nối, mở/đóng tab session, đồng bộ cấu hình, chỉnh desired state, xem trước và đẩy cấu hình, xem backup, duyệt database và tùy chỉnh ứng dụng.

3.2 Yêu cầu chức năng

Bảng 3.1: Yêu cầu chức năng và trạng thái hiện tại

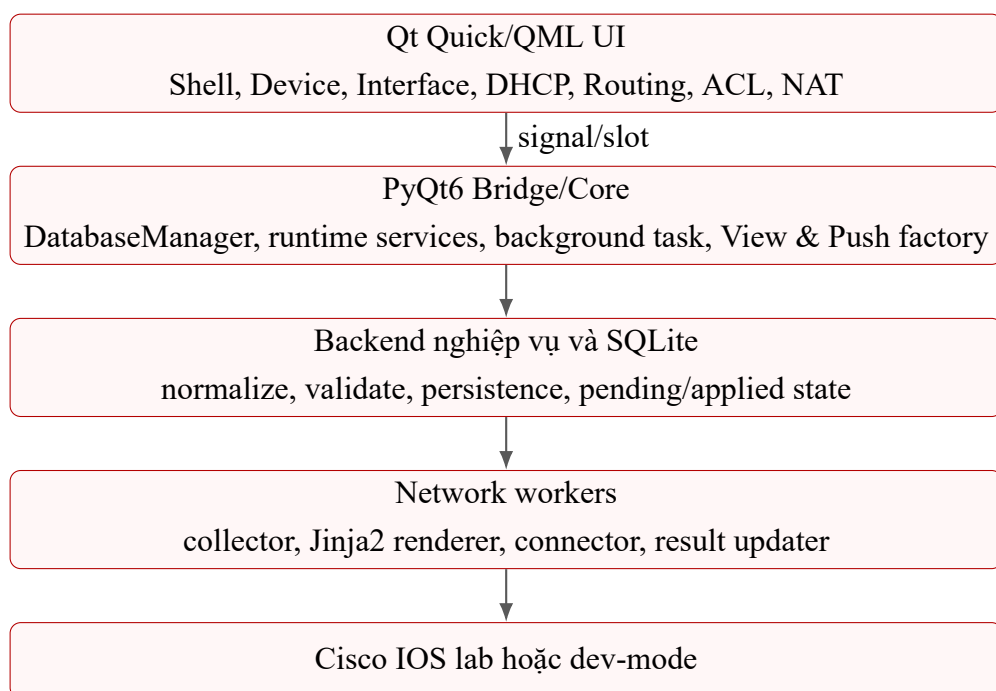
Mã	Yêu cầu	Trạng thái
FR-01	CRUD, tìm kiếm và nhập danh sách thiết bị từ JSON/XLSX	Đã triển khai
FR-02	Ping, kết nối, giữ session theo tab và backup running-config	Đã triển khai
FR-03	Đồng bộ interface và OSPF từ dữ liệu thu thập	Đã có nền tảng
FR-04	CRUD Interface L3/WAN/Tunnel/QoS	Đã triển khai
FR-05	View & Push Interface	Dự kiến
FR-06	CRUD và View & Push DHCP	Đã triển khai
FR-07	CRUD và View & Push Static/OSPF/EIGRP	Đã có nền tảng
FR-08	CRUD ACL nhiều loại và binding interface	Đã triển khai
FR-09	View & Push ACL	Dự kiến
FR-10	CRUD và View & Push NAT/PAT	Đã triển khai
FR-11	Database browser, external tools, theme và status settings	Đã triển khai

Mã	Yêu cầu	Trạng thái
FR-12	VLAN, BGP, VRF và topology automation	Dự kiến

3.3 Yêu cầu phi chức năng

- Giao diện nhất quán, tải lười các view lớn và giữ trạng thái form khi đổi tab.
- Tác vụ mạng dài chạy ở nền, có thông báo tiến trình và kết quả.
- Dev-mode không được mở session thật; nếu không xác minh được cờ dev thì phải fail-closed.
- Dữ liệu được kiểm tra định dạng, dùng khóa ngoại và giao dịch SQLite.
- Kiến trúc tách UI, bridge, persistence, worker và template.
- Password không được đưa vào log; mã hóa secret và phân quyền là yêu cầu tiếp theo.

3.4 Kiến trúc tổng thể



Hình 3.1: Kiến trúc runtime của NetworkTools

Entry point `app/main.py` nạp module QML UI và đưa các service Python vào context. `Data baseManager` là facade tổng hợp các mixin DHCP, ACL, NAT và các hàm thiết bị/routing.

3.5 Luồng dữ liệu cốt lõi

3.5.1 Quản lý và đồng bộ thiết bị

Người dùng tạo thiết bị, mở tab, ping hoặc kết nối. Connector lưu running-config theo host và parser đồng bộ một phần interface/OSPF vào database. Trạng thái tab quyết định có cho phép cấu hình hay không.

3.5.2 Lưu cấu hình mong muốn

Form QML kiểm tra dữ liệu cơ bản rồi gọi slot. Backend chuẩn hóa và ghi bản ghi với trạng thái pending. Thao tác xóa thường dùng soft delete để worker còn biết lệnh cần gỡ khỏi thiết bị.

3.5.3 View và Push

1. Controller thu thập bản ghi pending theo host/module.
2. Worker hoặc renderer tạo lệnh từ Jinja2 template.
3. Hộp thoại hiển thị preview; chưa có lệnh thì khóa nút Push.
4. Worker dùng connector của session hiện có hoặc xử lý dev-mode.
5. Báo cáo thành công được dùng để chuyển pending thành applied hoặc xóa bản ghi chờ gỡ.

3.6 Thiết kế cơ sở dữ liệu

Bảng 3.2: Phân nhóm schema SQLite

Tiền tố	Nội dung	Mức sử dụng
t01	Thiết bị và cấu hình truy cập/YANG	Runtime chính
t02	Interface router L3, tunnel, WAN, QoS	UI/persistence, chưa push
t03	DHCP pool, excluded, helper	Runtime chính
t04	Static route, OSPF, EIGRP	Runtime chính, còn lệch test schema
t05	ACL, binding, NAT, route-map	Runtime chính; ACL chưa push
t06	Switching L2	Schema dự phòng
t07	VRF	Schema dự phòng

Trường success thường mang ba trạng thái: 0 là pending thêm/sửa, 1 là đã áp dụng và -1 là pending remove/soft delete. Một số module còn dùng `action_Cfg` để mô tả phần cấu hình thay đổi.

3.7 Thiết kế giao diện

UI gồm shell, activity bar, sidebar thiết bị, feature bar, content area và status bar. Các view lớn được tạo bằng Loader ở lần truy cập đầu và giữ sống để tránh mất dữ liệu form. Nhóm form/list được tái sử dụng cho DHCP/NAT, còn OSPF/EIGRP sử dụng process card và các section chuyên biệt.

3.8 Thiết kế an toàn và xử lý lỗi

Các worker kiểm tra dev-mode trước khi yêu cầu session thật. Tác vụ mạng có kết quả cấu trúc và chỉ cập nhật database khi worker báo thành công. Những nội dung chưa hoàn chỉnh gồm mã hóa credential, audit log, verify sau push và rollback giao dịch nhiều thiết bị.

4 Xây dựng phần mềm

4.1 Môi trường phát triển

Ứng dụng yêu cầu Python 3.11 trở lên và sử dụng PyQt6, psutil, Netmiko, Nornir, ncclient, Requests, Jinja2, PyYAML, Scapy, PyShark và NAPALM theo khai báo trong `app/pyproject.toml`. Cơ sở dữ liệu là SQLite; runtime chính nằm trong thư mục `app/`.

4.2 Tổ chức mã nguồn

```
1 app/
2 |-- main.py           # entry point PyQt6/QML
3 |-- UI/               # QML, theme, reusable components, resources
4 |-- core/             # bridge, slots, runtime services, View & Push
5 |-- backend/          # validation and SQLite persistence
6 |-- network_code/     # connector, collectors, templates, workers
7 |-- database/         # modular schema and database builder
8 `-- tests/            # persistence, UI, QML and dev-mode tests
```

Listing 4.1: Cây thư mục runtime rút gọn

Thư mục `backend` của `kien/` chứa mã di sản/thử nghiệm. Do không phải module được `app/main.py` nạp, nó không được tính trực tiếp vào phạm vi sản phẩm đã hoàn thành.

4.3 Khởi tạo ứng dụng và QML bridge

`app/main.py` tạo `QApplication`, đăng ký QML module, khởi tạo database và đưa các đối tượng như `dbManager`, `cli`, `networkMonitor`, các settings và `AppPaths` vào context. QML không import trực tiếp file Python mà gọi các slot công khai.

4.4 Quản lý thiết bị, kết nối và đồng bộ

Module thiết bị hỗ trợ thêm, sửa, xóa, tìm kiếm và nhập từ JSON/XLSX. Mỗi host có thể tạo vùng backup riêng. Runtime hỗ trợ ping, mở external SSH client và kết nối/sync qua connector. Parser trong `network_code/login/sync_state.py` xử lý running-config, interface brief, interface block và OSPF để cập nhật SQLite.

4.5 Interface Router

Đã có nền tảng. Giao diện hỗ trợ L3, WAN, Tunnel và tùy chọn QoS; backend thực hiện upsert các bảng con và soft delete. Các slot `getRouterInterfaces`, `saveRouterInterface` và `deleteRouterInterface` là implementation thật. Tuy nhiên factory View & Push hiện chỉ đăng ký routing, DHCP và NAT, vì vậy Interface chưa có luồng triển khai end-to-end trong app chính.

4.6 DHCP

Đã triển khai. Ba tab Pool, Excluded và Helper sử dụng cơ chế staged save/cancel. Persistence chuẩn hóa subnet mask và theo dõi trạng thái thay đổi. `DhcpViewPushController` thu thập dữ liệu pending, render lệnh, gọi worker và cập nhật các hàng thành công.

4.7 Routing

4.7.1 Static route

Đã triển khai. Người dùng cấu hình default route hoặc nhiều static route, lưu desired state và dùng View & Push theo module static.

4.7.2 OSPF

Đã có nền tảng. UI có process, network, area, range, redistribute, passive interface, distance, tuning và interface settings. Backend có logic normalize/compare/store và worker có template/push. Bộ test hợp đồng hiện phát hiện backend còn truy cập tên bảng interface settings không tồn tại trong schema được test.

4.7.3 EIGRP

Đã có nền tảng. UI và backend bao phủ process, network, interface, passive interface, distribute list, offset list, redistribute và key chain. Lỗi hợp đồng schema tương tự OSPF cần được xử lý trước khi công bố hoàn thành.

Lưu ý phạm vi. Các Jinja2 template BGP đã tồn tại nhưng chưa có feature QML, bridge và persistence hoàn chỉnh. BGP không được tính là chức năng đã triển khai.

4.8 ACL

Đã có nền tảng. UI hỗ trợ Standard, Extended, Dynamic, Reflexive và MAC ACL, cùng binding một ACL lên nhiều interface. Backend đã có validation, persistence parent-child và soft delete; các test persistence tương ứng vượt qua. Tuy nhiên app chưa đăng ký ACL trong View & Push factory, nên chưa thể xem là luồng cấu hình thiết bị hoàn chỉnh.

4.9 NAT/PAT

Đã triển khai. Sáu nhóm giao diện gồm Static NAT, Dynamic NAT, PAT, NAT Interface, NAT ACL và Route Map. Backend lưu dữ liệu thật; collector gom bản ghi pending; template Cisco IOS tạo lệnh; worker và controller thực hiện preview/push rồi cập nhật trạng thái.

4.10 Tiện ích hệ thống

Ứng dụng có theme token, lưu trạng thái cửa sổ, status bar hiển thị mạng/RAM/ngày giờ, notification, database browser tích hợp, cấu hình external tools và background task. Các thành phần này hỗ trợ vận hành ứng dụng nhưng không thay thế việc xác minh cấu hình mạng.

4.11 Thành phần chưa tích hợp

Schema switching và VRF, BGP template, topology worker, FastAPI thử nghiệm, packet sniffer và các worker di sản chưa tạo thành luồng thống nhất trong runtime desktop. Chúng là nguồn tham khảo cho lộ trình phát triển, không phải kết quả hiện tại.

5 Thử nghiệm và đánh giá

5.1 Mục tiêu và nguyên tắc

Kiểm thử nhằm xác nhận persistence, hợp đồng QML–Python, khả năng tải giao diện, an toàn dev-mode và hành vi end-to-end trên lab. Số liệu trong báo cáo chỉ được cập nhật sau khi chạy trên cùng commit và ghi rõ môi trường.

5.2 Môi trường rà soát mã nguồn

Lần rà soát tài liệu được thực hiện trên Windows, dùng uv để chạy bộ test trong app/tests. Môi trường lab, image IOS và topology cần được nhóm điền sau khi chốt kịch bản thử nghiệm chính thức.

5.3 Kết quả kiểm thử tự động hiện tại

```
1 cd app
2 $env:UV_CACHE_DIR = ".uv-cache"
3 uv run python -m unittest discover -s tests -v
```

Listing 5.1: Lệnh chạy bộ kiểm thử

Bảng 5.1: Tổng hợp kết quả test tại thời điểm rà soát

Nhóm	Phát hiện	Đạt	Lỗi chức năng
Routing database contract	2	0	2
Dev-mode/worker safety	5	5	0
DHCP/ACL persistence	6	6	0
NAT persistence	6	6	0
QML smoke	4	4	0
UI/bridge contract	7	7	0
Tổng	30	28	2

Hai test OSPF và EIGRP thất bại vì backend truy cập `t04_ospf_interface_settings` và `t04_eigrp_interface_settings`, trong khi schema hiện hành dùng cấu trúc/tên bảng khác. Sau lỗi, Windows còn báo tệp SQLite tạm đang được giữ khi teardown. Đây là lỗi tích hợp cần sửa và chạy lại trước bản báo cáo cuối.

5.4 Kịch bản thử nghiệm lab đề xuất

Bảng 5.2: Kịch bản lab tối thiểu

Mã	Kịch bản	Mình chứng cần thu
LAB-01	Thêm thiết bị, ping, connect/sync	Backup running-config, interface/OSPF trong DB, log kết nối
LAB-02	DHCP	Client nhận địa chỉ; pool/excluded/helper đúng; trạng thái applied
LAB-03	Static route	show ip route, ping và lệnh preview
LAB-04	OSPF	Neighbor full, route học đúng, dữ liệu process/interface
LAB-05	EIGRP	Neighbor và route học đúng
LAB-06	NAT/PAT	show ip nat translations và lưu lượng qua được

Mã	Kịch bản	Minh chứng cần thu
LAB-07	Dev-mode	Không yêu cầu session thật; báo cáo mô phỏng có kiểm soát
LAB-08	Lỗi kết nối	App không treo, DB không bị đánh dấu applied sai

ACL và Interface chỉ đưa vào kịch bản push sau khi controller tương ứng được tích hợp. Các chức năng chỉ có schema như VLAN/VRF không được đưa vào bảng kết quả.

5.5 Phương pháp xác minh

Tùy module, sử dụng `show running-config`, `show ip interface brief`, `show ip route`, `show ip protocols`, `show ip dhcp binding`, `show ip nat translations` và `show access-lists`. `Ping/traceroute` được dùng để kiểm tra kết nối đầu cuối.

5.6 Đánh giá hiệu năng

Cần đo thời gian thao tác thủ công và bằng NetworkTools với 1, 5 và 10 thiết bị; thời gian preview/push; tỷ lệ thành công; CPU/RAM khi cần. Bảng sau được để trống có chủ đích, tránh sử dụng số liệu giả định.

Bảng 5.3: Biểu mẫu đo hiệu năng cần hoàn thiện

Thiết bị	Thủ công	NetworkTools	Giảm thời gian	Thành công
1	Chờ đo	Chờ đo	Chờ tính	Chờ đo
5	Chờ đo	Chờ đo	Chờ tính	Chờ đo
10	Chờ đo	Chờ đo	Chờ tính	Chờ đo

5.7 Đánh giá tổng hợp

Ưu điểm hiện tại là UI tập trung, cấu trúc module, persistence có test, bước preview, trạng thái pending/applied và dev-mode fail-closed. Hạn chế gồm phạm vi Cisco IOS, credential dạng rõ trong SQLite, parser phụ thuộc định dạng CLI, độ hoàn thiện không đồng đều, lỗi hợp đồng schema routing, chưa có rollback và chưa kiểm thử quy mô lớn.

6 Kết luận và hướng phát triển

6.1 Kết quả đạt được

Đề tài đã xây dựng nền tảng desktop PyQt6/QML kết hợp SQLite và các worker tự động hóa mạng. Các luồng hoàn thiện rõ nhất gồm quản lý thiết bị, DHCP, Static Routing và NAT/PAT; OSPF/EIGRP đã có UI/backend/worker nhưng còn lỗi hợp đồng schema; Interface và ACL mới hoàn thiện đến lớp giao diện và persistence.

Bảng 6.1: Đối chiếu mục tiêu và trạng thái

Mục tiêu	Trạng thái	Minh chứng
Khung ứng dụng và quản lý thiết bị	Đã triển khai	QML smoke, CRUD/import, tab/session
Thu thập và backup	Đã có nền tảng	Connector, parser interface/OSPF, file backup
DHCP View & Push	Đã triển khai	Controller, worker, persistence/dev-mode test
Routing View & Push	Đã có nền tảng	UI/backend/worker; 2 contract test còn lỗi
ACL	Đã có nền tảng	UI/persistence test; chưa có push controller
NAT/PAT View & Push	Đã triển khai	UI, persistence, collector, worker/controller
Switching, BGP, VRF, Dự kiến topology		Mới có schema/template/mã thử nghiệm

6.2 Ý nghĩa của đề tài

NetworkTools tạo môi trường thực hành kết hợp kiến thức mạng, lập trình Python, thiết kế giao diện và dữ liệu. Cơ chế desired state, preview và worker giúp sinh viên quan sát một quy trình tự động hóa có cấu trúc thay vì chỉ chạy chuỗi lệnh rời rạc.

6.3 Hạn chế

Hệ thống chưa mã hóa credential, chưa có phân quyền/audit đầy đủ, chưa triển khai rollback, chưa xác minh mọi biến thể Cisco IOS và chưa có số liệu quy mô lớn. Interface/ACL chưa push từ runtime chính. Hai contract test routing đang thất bại và phải được xử lý trước khi công bố bộ test hoàn chỉnh.

6.4 Hướng phát triển

1. Đồng bộ schema với backend OSPF/EIGRP và đưa toàn bộ test về trạng thái đạt.
2. Hoàn thiện View & Push cho Interface và ACL.
3. Bổ sung test worker NAT, verify sau push và thử nghiệm EVE-NG/GNS3 có thể tái lập.
4. Mã hóa secret, audit log, phân quyền và rollback.
5. Tích hợp VLAN/Switch Port, sau đó BGP và VRF.
6. Tích hợp topology discovery và static route đa router.
7. Mở rộng SFTP backup/versioning, Syslog, Serial Console, Firewall và plugin đa hãng.

Thứ tự trên ưu tiên độ tin cậy của chức năng hiện tại trước khi mở rộng số lượng module.

Tài liệu tham khảo

- [1] *Python Documentation*, Python Software Foundation, 2026. [Online]. Available: <https://docs.python.org/3/>
- [2] *Netmiko Documentation*, Netmiko Project, 2026. [Online]. Available: <https://ktbyers.github.io/netmiko/>

A Hướng dẫn cài đặt và chạy

A.1 Yêu cầu

Máy cần Python 3.11 trở lên và uv. Để dùng thiết bị thật, cần kết nối IP đến thiết bị và tài khoản quản trị phù hợp.

A.2 Khởi tạo và chạy ứng dụng

```
1 cd app
2 uv sync
3 uv run python main.py
```

Listing A.1: Cài dependency và chạy NetworkTools

Nếu database runtime chưa có, chạy builder theo hướng dẫn trong app/database/readme.md. Không dùng các file schema cũ ở thư mục khác để thay thế tùy ý vì có thể tạo sai hợp đồng bảng.

A.3 Chạy kiểm thử

```
1 cd app
2 $env:UV_CACHE_DIR = ".uv-cache"
3 uv run python -m unittest discover -s tests -v
```

Listing A.2: Chạy toàn bộ test với cache trong workspace

A.4 Biên dịch báo cáo

```
1 cd latex
2 ./build.ps1
```

Listing A.3: Biên dịch bằng XeLaTeX/latexmk

B Cấu trúc dự án và ánh xạ module

Đường dẫn	Vai trò
app/main.py	Entry point, khởi tạo PyQt6 và QML context
app/UI/	Giao diện QML, theme, component và resource
app/core/	Bridge, slot, runtime service, background task, View & Push
app/backend/	Validation và persistence cho DHCP, routing, ACL, NAT
app/network_code/	Login/sync, collector, template và worker
app/database/	Schema module, database builder và tài liệu dữ liệu
app/tests/	Test persistence, worker, UI contract và QML smoke
latex/	Nguồn báo cáo NCKH và script build
backend của kien/	Mã di sản/thử nghiệm, không phải runtime desktop chính

B.1 Ảnh xạ chức năng chính

Chức năng	UI/bridge	Backend/worker
DHCP	UI/qml/dhcp, core/dhcp_slots.py	backend/dhcp, network_code/dhcp
Routing	UI/qml/routing, core/databas e.py	backend/route, network_code/routing
ACL	UI/qml/acl, core/acl_slots.py	backend/acl; chưa có worker runtime chính
NAT	UI/qml/nat, core/nat_slots.py	backend/nat, network_code/nat
Interface	UI/qml/interface, core/dhcp_slots.py	backend/dhcp/interfaces.py; chưa có View & Push

C Danh sách kiểm thử và minh chứng cần hoàn thiện

C.1 Nhóm test tự động

- `test_database_routing_contract.py`: hợp đồng OSPF/EIGRP với schema.
- `test_dev_mode_workers.py`: an toàn dev-mode và cập nhật trạng thái.
- `test_dhcp_acl_persistence.py`: DHCP/ACL CRUD, validation và binding.
- `test_nat_persistence.py`: các hình thức NAT/PAT và soft delete.
- `test_qml_smoke.py`: tải QML module, view và input harness.
- `test_ui_contracts.py`: hợp đồng form, slot và trạng thái cửa sổ.

C.2 Minh chứng cần chèn vào bản cuối

1. Ảnh tổng quan app và từng module có dữ liệu thống nhất.
2. ERD rút gọn đúng schema hiện hành.
3. Topology lab, bảng địa chỉ và image IOS.
4. Log test sau khi sửa lỗi routing schema.
5. Preview và output lệnh show trước/sau push.
6. Bảng đo thời gian, tỷ lệ thành công và môi trường chạy.